HANIKA ANGLICAN INTEGRATED POLYTECHNIC (HAIP) · Data Mining and Data Warehousing · FA1 Group Work

Backup and Recovery Plan

Strategy, procedures, disaster recovery and restore test

Project: University Library Analytics Data Warehouse

Group: **Group X** (replace with your group number)

Members: Member 1 (Data Architect) · Member 2 (ETL Specialist) · Member 3 (Analytics & Security Lead)

Database: SQLite 3.53.1 · ETL: Python 3 + pandas · Dashboards: Power BI Desktop

Submission: November 2, 2026 · Presentation: November 6, 2026

Contents

1. What must be protected
2. Backup strategy: full + differential + incremental
3. Restore procedures
    - 3.1 Restore the newest state (most common)
    - 3.2 Restore an older full backup
    - 3.3 After the restore
4. Disaster recovery plan
5. Restore test (how we proved it works)
    - 5.1 Automatic restore test
    - 5.2 Disaster-recovery drill
6. Scalability of the backup design

1. What must be protected

Asset	Size	Can it be rebuilt?
`library_dw.db` (staging + warehouse + security + audit)	2,976 KB	partly: the warehouse can be rebuilt by the ETL from the source exports, but users, audit log and history cannot
Source exports (`09_Source_Data`)	small	kept by the source systems
Scripts and documents	small	in the Git repository (GitHub)
Backup encryption key `backup.key`	44 bytes	**no** - keep 2 copies off the server

Targets

	Target	How
RPO (maximum data loss)	4 hours during opening hours, 24 hours at night	incremental backup every 4 hours, differential every night
RTO (maximum downtime)	1 hour (measured restore: under 1 second + checks)	scripted restore `backup_restore.py restore latest`

2. Backup strategy: full + differential + incremental

Type	What it contains	When	Kept
Full	a complete, consistent copy of the database file (SQLite online backup API - users can keep working), integrity-checked, SHA-256 checksum, **AES-encrypted**	every Sunday 01:00	4 weeks (28 days), newest always kept
Differential	everything changed since the last **full** backup (new ETL batches, new audit and error-log rows), as an SQL file	every night after the ETL (00:30 job)	until the next full
Incremental	everything changed since the **last backup of any kind**	every 4 hours, 08:00-20:00	until the next full

Storage (3-2-1 rule): 3 copies of the data, on 2 different media, 1 off-site:

1. the live database on the library server;
2. the `backups/` folder on a second disk or network share (set `BACKUP_DIR` in the script);
3. a weekly copy of the encrypted full backup to the university's cloud storage (encrypted, so the provider cannot read it).

Every backup is listed in `backups/backup_catalog.json` (type, file, time, checksum, row counts).

Scheduling: `schedule_backups.bat` (run once as administrator) creates three Windows Task Scheduler jobs: *LibraryDW Nightly ETL* (00:30, ETL + differential), *LibraryDW Full Backup* (Sunday 01:00), *LibraryDW Incremental Backup* (every 4 hours from 08:00).

3. Restore procedures

3.1 Restore the newest state (most common)

```
cd 07_Security_Documentation\backup_scripts
python backup_restore.py restore latest
```

The script: (1) saves the current (damaged) database as `library_dw_before_restore_<time>.db`; (2) checks the **SHA-256 checksum** of every backup file; (3) decrypts the newest full backup; (4) applies the newest **differential** and then every later **incremental**; (5) runs `PRAGMA integrity_check`; (6) only if everything is OK, replaces the database file.

3.2 Restore an older full backup

```
python backup_restore.py list
python backup_restore.py restore library_dw_full_20261101_010000.db.enc
```

3.3 After the restore

1. `python backup_restore.py verify` - compares every table.
2. Open the Executive dashboard and compare the totals with the last known report.
3. Run the ETL once if the restored state is older than the last source exports (the ETL reload brings the warehouse fully up to date).
4. Write a short incident note: cause, time of failure, time restored, data lost.

4. Disaster recovery plan

Disaster	Response	Time
Database file deleted or corrupted	`restore latest` (section 3.1)	< 15 min
Bad data loaded by the ETL	rerun the ETL after fixing the source (full reload) or `restore` the backup taken before the batch	< 30 min
Server disk failure	new disk / laptop → install Python → copy the project from GitHub → copy the backups from the share → `restore latest`	< 1 hour
Ransomware / server lost	as above on a clean machine, using the **off-site** encrypted copy and the key from the safe	< 4 hours
Backup key lost	old encrypted backups cannot be opened → immediately make a new full backup with a new key	-

Roles: the *admin* performs the restore; the *ETL specialist* checks the data; the *Library Director* decides when the system is back in service and informs the users. The plan is **tested every 3 months** (section 5) and after every change to the database structure.

5. Restore test (how we proved it works)

5.1 Automatic restore test

`python backup_restore.py verify` restores the newest backup chain into a scratch file and compares **every table** and the business totals with the live database:

```
======================================================================
 BACKUP RESTORE TEST
======================================================================
 restored from : library_dw_full_20261026_193856.db
 integrity     : ok
 restore time  : 0.02 s

 table                         live  restored  result
 audit_log                       29        29  PASS
 dim_book                        73        73  PASS
 dim_booking_purpose              7         7  PASS
 dim_date                      1827      1827  PASS
 dim_department                   9         9  PASS
 dim_resource                    47        47  PASS
 dim_room                        12        12  PASS
 dim_student                    801       801  PASS
 dim_time_slot                    7         7  PASS
 dim_user_type                    5         5  PASS
 etl_batch_log                    1         1  PASS
 etl_error_log                  527       527  PASS
 fact_book_loan                2205      2205  PASS
 fact_digital_usage            4023      4023  PASS
 fact_room_booking             2837      2837  PASS
 map_department                  27        27  PASS
 map_room                         3         3  PASS
 map_time_slot                   21        21  PASS
 sec_role_permissions            18        18  PASS
 sec_roles                        5         5  PASS
 sec_users                        9         9  PASS
 stg_book_catalog                72        72  PASS
 stg_book_transactions         2278      2278  PASS
 stg_digital_usage             4113      4113  PASS
 stg_room_bookings             2999      2999  PASS
 stg_student_master             800       800  PASS

 total loans / downloads: live (2205, 18623), restored (2205, 18623)  PASS

 RESULT: PASS - the backup can be restored completely

======================================================================
```

5.2 Disaster-recovery drill

We made an encrypted full backup, let users work (security tests wrote audit rows), took an incremental and a differential backup, then **deleted the database file** and restored it:

```
= DISASTER-RECOVERY DRILL  (2026-10-26 19:39)
======================================================================
> python backup_restore.py full --encrypt
  new encryption key saved in backup.key - store a copy OFF this computer
FULL backup OK: backups/full/library_dw_full_20261026_193903.db.enc  (3957 KB, 0.1 s,
integrity ok, encrypted)
> (users work: python secure_access.py --demo)
> python backup_restore.py incremental
INCREMENTAL backup OK: backups/incremental/library_dw_incremental_20261026_193903.sql  (29
changed rows)
> python backup_restore.py differential
DIFFERENTIAL backup OK: backups/differential/library_dw_differential_20261026_193903.sql
(29 changed rows)
> python backup_restore.py list
2026-10-26T19:38:56  full           2968 KB
backups/full/library_dw_full_20261026_193856.db
2026-10-26T19:39:03  full           3957 KB
backups/full/library_dw_full_20261026_193903.db.enc
2026-10-26T19:39:03  incremental       8 KB
backups/incremental/library_dw_incremental_20261026_193903.sql
2026-10-26T19:39:03  differential      8 KB
backups/differential/library_dw_differential_20261026_193903.sql

> DISASTER: library_dw.db is deleted
  ls: cannot access '../../03_Database_Files/library_dw.db': No such file or directory
> python backup_restore.py restore latest
RESTORE OK from ['library_dw_full_20261026_193903.db.enc',
'library_dw_differential_20261026_193903.sql'] in 0.0 s
> python backup_restore.py verify
```

After the restore, `verify` reported **PASS for all 26 tables** (2,205 loans, 18,623 downloads, all users, all audit rows). Full output: `backup_scripts/backup_test_results.txt`.

6. Scalability of the backup design

- The full backup of today's database takes about 0.1 s; even at 100× the data the weekly full backup stays under a minute.
- If the library moves to MySQL/PostgreSQL (many simultaneous users), the same strategy maps directly: full = `mysqldump` / `pg_dump`, incremental = binary log / WAL archiving, same schedule and restore test.